

บทที่ 3

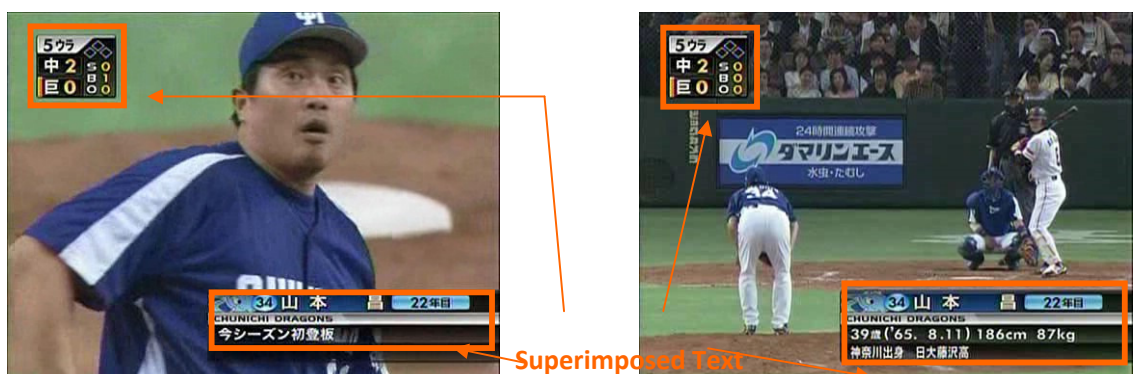
การค้นหาจากวิดีโอ โดยอ้างอิงจากผู้ใช้

3.1 การค้นหาจากในวิดีโอ อาศัย Superimposed text เป็นหลักในการค้นหา

3.1.1 แนวความคิด

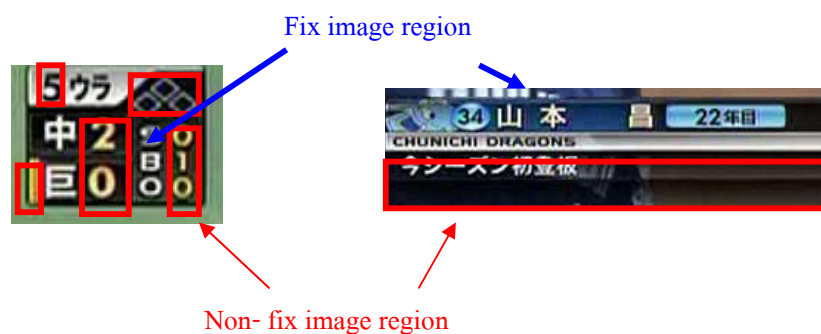
การทำดัชนีของวิดีโอ นั้น ส่วนใหญ่จะเริ่มจากการจดจำข้อความบนวิดีโอ (Superimposed Text) ซึ่งนับว่าเป็นข้อจำกัดอย่างยิ่งในการทำระบบออกไปใช้งานในที่ที่ภาษานั้นต่างกัน เช่น ระบบที่ออกแบบในประเทศไทย ก็ยากจะนำไปใช้ในประเทศญี่ปุ่น ซึ่งจุดนี้เองทำให้ผู้พัฒนาคิดถึงระบบแบบ World Wide (ทั่วโลก) ที่สามารถใช้ร่วมกันได้

Superimposed Text เป็นข้อความที่ใช้อธิบายเหตุการณ์ หรือ สรุปเหตุการณ์ในเฟรมวิดีโอ ณ.เวลานั้นๆ ซึ่งจะถูกสร้างมาจากผู้กำกับ หรือผู้จัดทำในสตูดิโอของห้องส่ง ซึ่งข้อความเหล่านี้จะสั้น และกระชับ มีความหมายชัดเจน เช่น สกอร์ของการแข่งขันฟุตบอล การแสดงชื่อผู้เล่นเมื่อกล้องจับหน้าผู้เล่นคนนั้น หรืออธิบายเกม ก็ตาม ซึ่งจะเป็นส่วนดึงดูดผู้ใช้ให้สนใจมากขึ้น ซึ่งจากการสังเกตรูปแบบวิดีโอ นั้น Superimposed Text นับเป็นคีย์หลัก เพราะว่า พื้นที่ของ Superimposed Text (ภาพที่ 3.1) นั้นจะมีตำแหน่งคงที่ และภายในพื้นที่นั้นจะมี พื้นที่. Fix-image region โดยยกตัวอย่างเช่นภาพที่ 3.2 การนำจุดนี้มาดัดแปลงเป็นดัชนีให้กับผู้ใช้นั้น พบว่า “ระบบสามารถใช้ได้แบบ World Wide โดยไม่ต้องอาศัยการจดจำ”



รูปที่ 3.1 แสดง Superimposed Text

ซึ่ง ณ. ตำแหน่งเดียวกัน Superimposed Text จะมีเนื้อหาเป็นเรื่องเดียวกัน เช่น คะแนน ในการแข่งขันเบสบอล ณ. ประเทศญี่ปุ่น จะปรากฏอยู่ทางซ้ายมือด้านบนเสมอ และ จะมีข้อความอธิบายผู้เล่น ในด้านขวามือล่าง ที่แสดงให้เห็นในภาพที่ 3.1



รูปที่ 3.2 แสดง Non – fix image region

จากการสังเกต พบว่าความสัมพันธ์ระหว่าง พื้นที่.คงที่ กลับ พื้นที่. ไม่คงที่ นั้นสามารถนำมาทำดังนี้ได้ โดยมีความสัมพันธ์ดังนี้

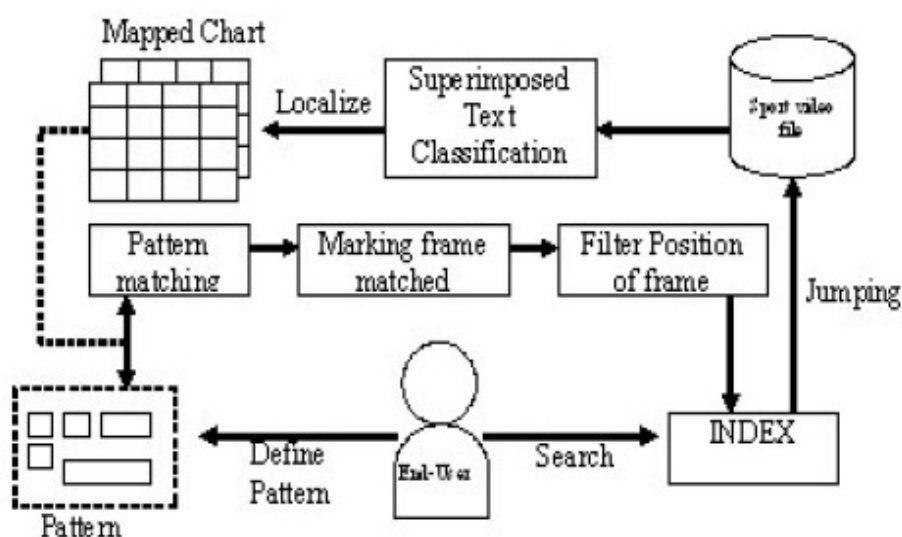
1. พื้นที่คงที่ และ พื้นที่ไม่คงที่ จะวิเคราะห์ภายใน Superimposed Text เท่านั้น
2. พื้นที่.คงที่ จะปรากฏ ณ. ตำแหน่งเดียวกัน มีสีที่เหมือนกันทั้งตลอดรายการ หรือ ทั้งเกมกีฬา โดยพื้นที่ ที่ไม่คงที่จะเปลี่ยนแปลง เช่น คะแนน ชื่อ รายละเอียด
3. Superimposed Text จะเป็นตัวอธิบายเหตุการณ์ ณ. ขณะเฟรมนั้น ปรากฏอยู่และมักจะขึ้น เมื่อเกิดเหตุการณ์สำคัญๆ เช่น ในเกมกีฬา; มีการเปลี่ยนตัวผู้เล่น มีอธิบายลักษณะของแต่ละผู้เล่น เกม หรือคะแนนในการแข่งขัน เป็นต้น

จากข้อสันนิษฐานทั้งสามข้อนี้ ผู้จัดทำได้ออกแบบ ลักษณะตัวโปรแกรม โดยระบบที่ออกแบบมานั้น จะอนุญาตให้ผู้ใช้ นั้น สามารถกำหนดรูปแบบขึ้นมาเองได้ (Pattern) ซึ่งพบว่า “การกำหนดรูปแบบโดยผู้ใช้นั้น จะตรงกับความต้องการของตนเองมากที่สุด [4]”

3.1.2 โครงสร้างของระบบ

การออกแบบ ผู้จัดทำได้ออกแบบ ระบบการค้นหาจากแบบ World Wide และอนุญาตให้ผู้ใช้ได้กำหนดรูปแบบในการค้นหาเอง โดยทำการพัฒนาโดยใช้เครื่องมือ VC++.net กับ OpenCV library โดยการทดลองจะทดลองกับกีฬา เบสบอล ซึ่งจะเห็นภาพชัดมากที่สุด เพราะ กีฬาเบสบอลมีรายละเอียดในการเล่นมากมาย

ระบบนั้น จะเน้นว่า ทำอย่างไรให้ติดต่อกับผู้ใช้งานได้ง่ายที่สุด โดยให้ผู้ใช้งานได้กำหนดรูปแบบได้งานที่สุด และ ผลลัพธ์ออกมาได้ตรงกับความต้องการของผู้ใช้งานจริง



รูปที่ 3.3 โครงสร้างของระบบค้นหาในไฟล์วิดีโอ

ในการออกแบบให้ผู้ใช้เลือก Pattern จะทำการจำลองว่า ผู้ใช้งานนั้นได้ใช้รีโมท เป็นตัวออกแบบ ไม่ใช่ในเมาส์หรือคีย์บอร์ดเป็นตัวออกแบบ โดยวิธีการใช้รีโมทนั้น จะไม่สามารถที่จะกำหนดรูปแบบได้โดยละเอียด โดยผู้จัดทำได้ออกแบบให้ผู้ใช้งานนั้นออกแบบรูปแบบ ได้สองวิธี คือ

1. เลือกรูปแบบในการค้นหาโดยกำหนด พื้นที่ คงที่ไว้
2. เลือกรูปแบบโดยใช้เครื่องมือช่วยเหลือ หาพื้นที่ คงที่



รูปที่ 3.4 การเลือกรูปแบบการค้นหาวีธีที่ 1



รูปที่ 3.5 การเปรียบเทียบหาพื้นที่คงที่

วิธีที่สอง นั้นเป็นการใช้ตัวช่วยในการกำหนด พื้นที่คงที่ โดยเราจะเลือกเพียงแค่ Superimposed Text จากหลายๆ จาก แล้วนำมารวมกัน เพื่อหา พื้นที่คงที่ภายในโดยอาศัยหาพิกเซลที่เคลื่อนที่ หรือ พิกเซลที่เปลี่ยนไป เช่น ภาพที่ 3.5 ซึ่งจะเป็นการหาพื้นที่คงที่ จากฉากที่สามโดย พื้นที่. ในกรอบสีน้ำเงินนั้น จะเป็นพื้นที่. ที่เราไม่ต้องการ ใช้ในการค้นหา พิกเซลที่มีการเปลี่ยนแปลงนั้น จะใช้ หลักการของ Standard Deviation

$$M_{(x,y)} = \frac{S_{(x,y)}}{N} \quad (3.1)$$

$$\sigma_{(x,y)} = \sqrt{\frac{Sq_{(x,y)}}{N} - \left(\frac{S_{(x,y)}}{N}\right)^2} \quad (3.2)$$

$$|M_{(x,y)} - P_{(x,y)}| > C \cdot \sigma_{(x,y)} \quad (3.3)$$

- $S_{(x,y)}$ คือค่าผลบวกของค่า pixel ในแต่ละตำแหน่ง
- $Sq_{(x,y)}$ คือค่าผลบวกของกำลัง 2 ของค่าในตำแหน่ง
- N คือค่าตัวเลขของ frame ที่รวบรวมไว้
- C คือค่าคงตัว
- $\sigma_{(x,y)}$ คือค่าการเบี่ยงเบนมาตรฐาน
- $P_{(x,y)}$ คือค่าในตำแหน่ง pixel

สาเหตุของการใช้ Standard Deviation นั้นก็ ค่าพิชเชลนั้นในแต่ละตำแหน่งจะมีการกระจายของค่าความสว่างอยู่ ซึ่งเราไม่สามารถนำมาลบกันได้ ก็คือ แม้ว่าจะเป็นพิชเชลที่ตำแหน่งเดียวกัน รูปเหมือน แต่ค่าความสว่างของพิชเชลอาจจะต่างกัน ดังนั้น เราไม่สามารถนำรูปแต่ละรูปมาลบกันได้โดยตรง เราต้องการหา Standard Deviation เสียก่อน ซึ่งจะนำมาใช้ได้ว่า พิชเชลนั้น ๆ ได้เปลี่ยนแปลงไปหรือไม่ ตาม สมการที่ (3.1), (3.2) และ (3.3) โดยที่ พิชเชลที่อยู่ในเงื่อนไขที่ (3.3) นั้นถือว่าเป็นพิชเชลที่เคลื่อนที่ และ เป็นพิชเชลที่เปลี่ยนแปลง โดยผู้จัดทำได้ทำการทดลอง กับ สกอร์บอร์ด ใน กีฬาเบสบอล ดังภาพที่ 3.6

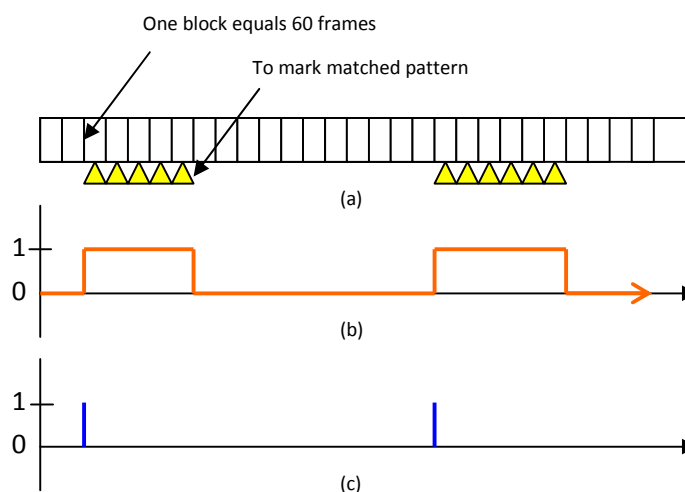


รูปที่ 3.6 การหาพิชเชลที่เคลื่อนที่

ระบบจะทำการนำรูปแบบของผู้ใช้ที่ได้ออกแบบไว้ มาเปรียบเทียบกับวิดีโอ ซึ่งการเปรียบเทียบนั้น จะเป็นไปตามสมการ ที่ (3.4)

$$I = \begin{cases} 1; & \sum_{[i,j] \in R} (f - g)^2 < Threshold \\ 0; & otherwise \end{cases} \quad (3.4)$$

โดยที่ f เป็นภาพของ Pattern และ ภาพ g เป็นภาพที่จะทำการเปรียบเทียบในวิดีโอไฟล์ ซึ่งได้กำหนดไว้ว่า ถ้าภาพที่ผ่าน threshold จะกำหนดไว้เป็น 1 และภาพที่ไม่ผ่านจะกำหนดไว้เป็น 0 โดยการเปรียบเทียบนั้น จะไม่ได้เปรียบเทียบ pattern กับภาพทั้งเฟรม จะเปรียบเทียบเฉพาะตำแหน่งเดียวกันกับ pattern



รูปที่ 3.7 การระบุตำแหน่งเฟรมที่ตรงกับ Pattern

เฟรมที่ตรงกัน กับ สมการที่ (4) จะทำการกำหนดตำแหน่งไว้ (Marking) ซึ่งพบว่า เราจะไม่ทำการเปรียบเทียบทุกๆ เฟรม ซึ่งจะทำการเปรียบเทียบ ทุก ๆ 30 เฟรมแทน เพราะว่า ไม่มีประโยชน์ที่จะทำการเปรียบเทียบทุกๆ เฟรม ยิ่งทำให้เสียเวลามากขึ้น เพราะ ในหนึ่งวินาที ภาพนั้น เกิดการเปลี่ยนแปลงทางเนื้อหาบ่อยมาก จากนั้น จะทำการลดทอน กราฟออกแบบ ดังรูป ที่ 3.7 (b) และ จะกำหนดจากที่ตรงกับรูปแบบของผู้ใช้ โดยกำหนดไว้ในขอบข่ายของกราฟ ดังภาพที่ 3.7 (c)

3.2 การพัฒนาโปรแกรมจำลองการค้นหาคอนวินด์โอ

3.2.1 วิธีการพัฒนาโปรแกรม

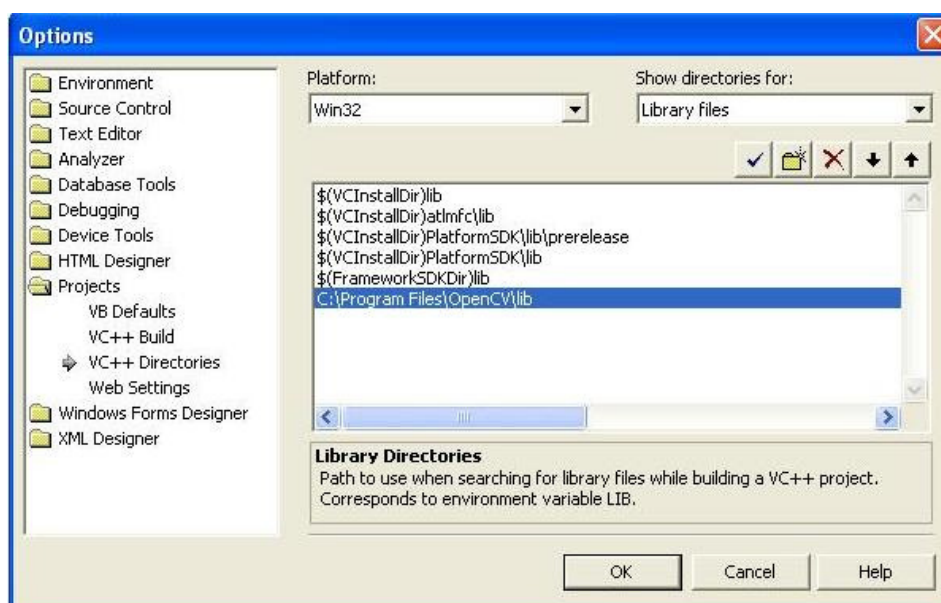
การพัฒนาโปรแกรมได้ใช้เครื่องมือ คือ Visual C++.net ร่วมกับ OpenCV library ในการพัฒนาโปรแกรมจำลองการค้นหาคอนวินด์โอ โดยขั้นตอนการพัฒนาเริ่มต้นจาก

1. ติดตั้งค่า Global เพื่อเชื่อมการใช้งาน OpenCV และ VC++.net

หลังจากการติดตั้ง OpenCV ที่ได้ดาวน์โหลดมาจากเว็บ source force แล้วนั้น ก็ต้องทำการตั้งค่าก่อนโดยได้อธิบายเป็นลำดับดังต่อไปนี้

- เปิด Visual C++.Net เลือก Tools-> Options ในเมนูบาร์
- เลือก Project -> VC++ Directories ในลิสต์
- ลำดับแรกนั้น เลือก Library files จาก “Show Directories for” ใน list box
- คลิกไอคอน Insert New และเลือก folder ที่ได้ทำการติดตั้ง OpenCV
- ในไลบรารี ให้เพิ่ม

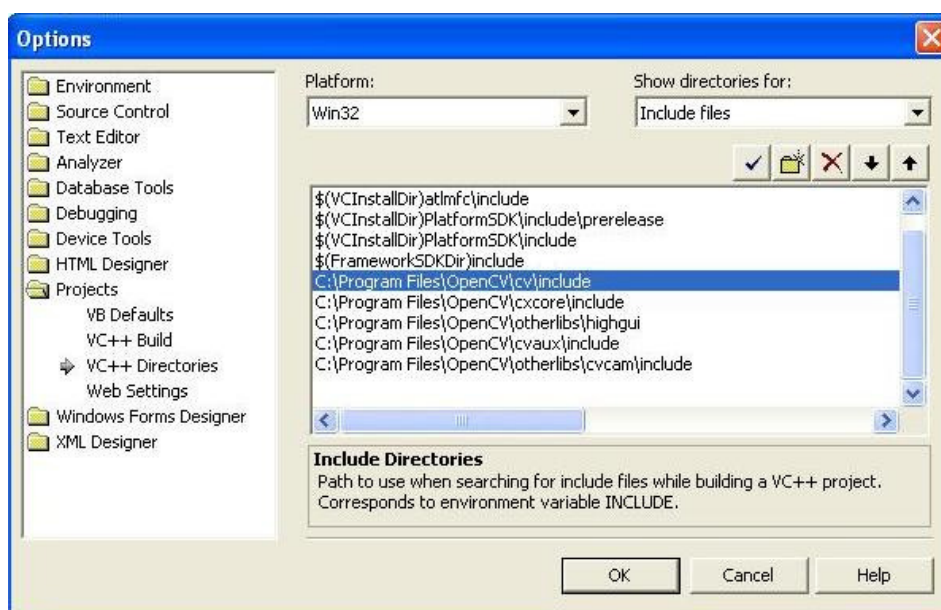
"C:\Program Files\OpenCV\lib"



รูปที่ 3.8 การตั้ง path ไลบรารีให้สามารถใช้งานร่วมกับ OpenCV

ต่อไปทำการเลือกหัวข้อต่อมา คือ Include file ภายใน list box และกำหนด path เช่นเดียวกับไลบรารี โดยเลือกใดเรกทอรีดังต่อไปนี้

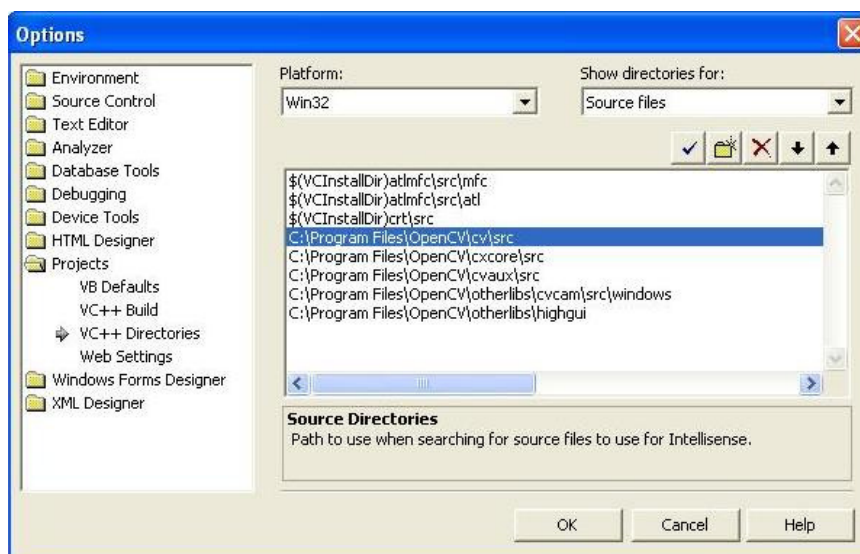
```
"C:\Program Files\OpenCV\cv\include"
"C:\Program Files\OpenCV\cxcore\include"
"C:\Program Files\OpenCV\otherlibs\highgui"
"C:\Program Files\OpenCV\cvaux\include"
"C:\Program Files\OpenCV\otherlibs\cvcam\include"
```



รูปที่ 3.9 การตั้ง path Include file ให้สามารถใช้งานร่วมกับ OpenCV

ต่อมาเลือก Source files ใน list box และทำการตั้งค่า path ไดรกทอรี เช่นเดียวกับที่ทำมาในขั้นต้น

```
"C:\Program Files\OpenCV\cv\src"
"C:\Program Files\OpenCV\cxcore\src"
"C:\Program Files\OpenCV\cvaux\src"
"C:\Program Files\OpenCV\otherlibs\highgui"
"C:\Program Files\OpenCV\otherlibs\cvcam\src\windows"
```

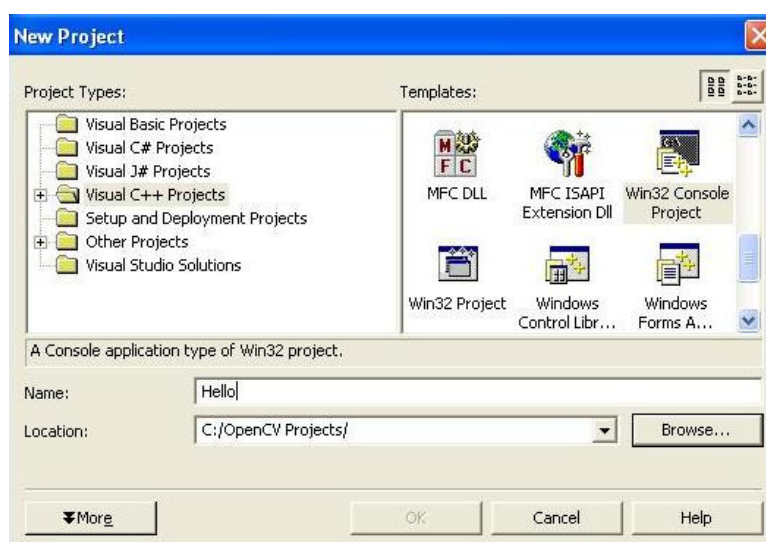



รูปที่ 3.10 การตั้ง path source file ให้สามารถใช้งานร่วมกับ OpenCV

จากนั้นเลือก Ok ที่หมดนี้เป็นขั้นตอนการติดต่อค่าเริ่มต้นชนิด Global Setting เพื่อลิงก์การใช้งานร่วมกันระหว่าง OpneCV และ Visual C++.net

2. สร้างโปรเจกใหม่

ในเครื่องมือการพัฒนาของ Studio นั้น ให้สร้างโปรเจกใหม่ โดยการเลือกเมนู File -> New...->"Project" โดยทำการเลือก "Win32 Application" หรือ "Win32 console application"



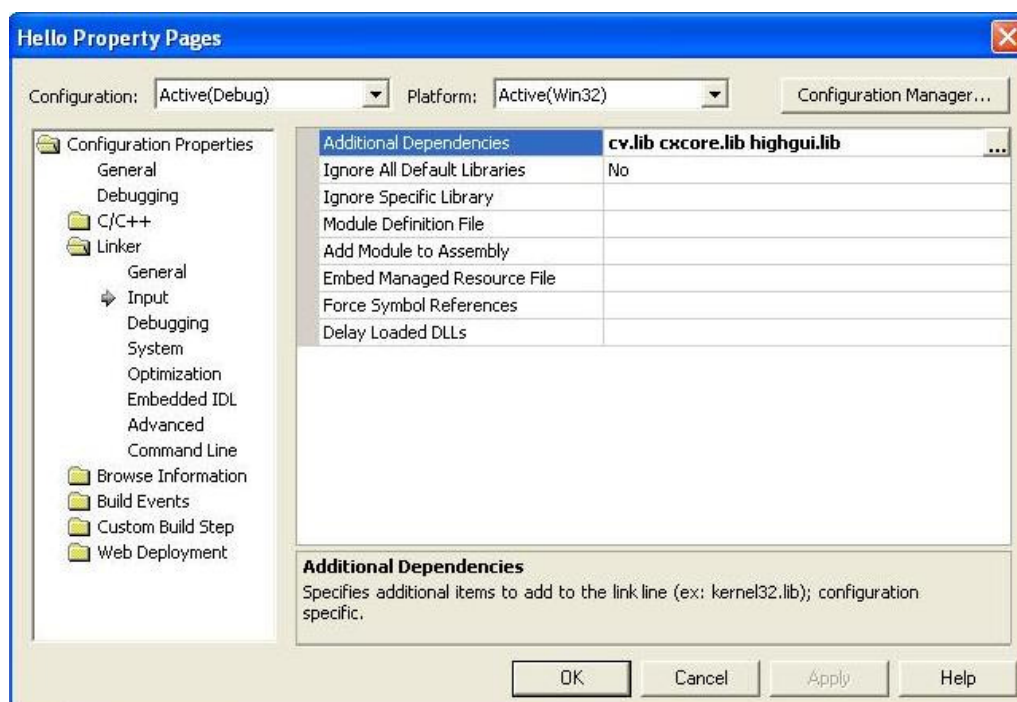
รูปที่ 3.11 การสร้างโปรเจกใหม่

หลังจากเสร็จสิ้นขั้นตอนเบื้องต้นนั้น Developer Studio จะสร้างขึ้นมา 5 ไฟล์ นั่นก็คือ <project name>.vcproj , <project name>.sln, <project name>.cpp, stdafx.cpp และ stdafx.h ซึ่ง StdAfx นี้เองนั้นเป็นไฟล์ header ก่อนที่จะเกิดการคอมไพล์ ซึ่งเป็นประโยชน์มากในกรณีลดเวลาในการคอมไพล์ตัวโปรแกรม

3. วิธีการเขียนโปรแกรมร่วมกัน OpenCV

เปิดไฟล์ <project name>.cpp จากนั้นเพิ่มคำสั่งที่เกี่ยวข้องกับ Open คือ #include<cv.h>, #include<cxcore.h>, #include<highgui.h> ควรจะเพิ่มหลังจาก stdafx.h ไม่งั้นจะเกิดการ build error ได้ จากนั้นก็ต้องมีการติดตั้งค่าเฉพาะในโปรเจก คือ เลือกเมนู “Project”-> “Properties” เลือก “Linker” ในแท็บ -> “Input” และหัวข้อ -> “Additional Dependencies:” จากนั้นเพิ่ม path ซึ่ง path ที่จำเป็นในโครงการนี้ คือมีการโหลดไลบรารี คือ

cxcore.lib, cv.lib, highgui.lib, cvaux.lib และ cvcam.lib



รูปที่ 3.11 ตัวอย่างการเพิ่มไลบรารีในโปรเจก

4. การเข้าถึงภาพในวิดีโอ (Image Access)

ในการเข้าถึงภาพนั้น จะใช้ `IplImage` class ของไลบรารี ซึ่งเป็นชนิดของภาพที่ใช้ใน IPL และ OpenCV API นับว่าเป็นฟังก์ชันพื้นฐานที่ใช้สำหรับการเก็บชนิดของข้อมูลภาพ ถูกเก็บไว้ใน `ipl.h` ในไลบรารี OpenCV นั้นเสนอให้ผู้พัฒนาได้สามารถสร้างฟังก์ชันได้โดยง่ายและสะดวก ซึ่งโครงสร้างของ `IplImage` มีรูปแบบดังนี้คือ

```
typedef struct _IplImage
{
    int nSize;          /* sizeof(IplImage) */
    int ID;             /* version (=0) */
    int nChannels;       /* Most of OpenCV functions support 1,2,3 or 4 channels */
    int alphaChannel;    /* ignored by OpenCV */
    int depth;          /* pixel depth in bits: IPL_DEPTH_8U, IPL_DEPTH_8S, IPL_DEPTH_16U,
                        IPL_DEPTH_16S, IPL_DEPTH_32S, IPL_DEPTH_32F and IPL_DEPTH_64F are supported */
    char colorModel[4]; /* ignored by OpenCV */
    char channelSeq[4]; /* ditto */
    int dataOrder;      /* 0 - interleaved color channels, 1 - separate color channels.
                        cvCreateImage can only create interleaved images */
    int origin;         /* 0 - top-left origin,
                        1 - bottom-left origin (Windows bitmaps style) */
    int align;          /* Alignment of image rows (4 or 8).
                        OpenCV ignores it and uses widthStep instead */
    int width;          /* image width in pixels */
    int height;         /* image height in pixels */
    struct _IplROI *roi; /* image ROI. when it is not NULL, this specifies image region to process */
    struct _IplImage *maskROI; /* must be NULL in OpenCV */
    void *imageId;      /* ditto */
    struct _IplTileInfo *tileInfo; /* ditto */
    int imageSize;      /* image data size in bytes
                        (=image->height*image->widthStep
                        in case of interleaved data) */
    char *imageData;    /* pointer to aligned image data */
    int widthStep;      /* size of aligned image row in bytes */
    int BorderMode[4]; /* border completion mode, ignored by OpenCV */
    int BorderConst[4]; /* ditto */
    char *imageDataOrigin; /* pointer to a very origin of image data
                        (not necessarily aligned) -
                        it is needed for correct image deallocation */
}
IplImage;
```

รูปที่ 3.12 โครงสร้างของ `IplImage` ในไลบรารี OpenCV

ในการเข้าถึงวิดีโอ นั้น ทาง OpenCV จะมีฟังก์ชันโดยเริ่มจากการเปิดไฟล์วิดีโอคือ

```
CvCapture *input_video = cvCaptureFromFile("filename.avi");
```

จากนั้น ในการ access เข้าถึงเฟรมวิดีโอและการ get property ของวิดีโอ ประกอบด้วย หมายเลขเฟรม , ขนาดของเฟรม เป็นต้น

```
cvQueryFrame( input_video );
IplImage *Getframe = cvQueryFrame( input_video );
long Number_fame = cvSetCaptureProperty
                    (input_video,CV_CAP_PROP_FRAME_COUNT);
CvSize frame_size;
Frame_size.height =
cvGetCaptureProperty(input_video,CV_CAP_PROP_FRAME_HEIGHT);
```

5. การแสดงผลวิดีโอ

เนื่องจากทางผู้พัฒนานั้นต้องการที่จะให้ภาพวิดีโอนั้นขึ้นในตัว Application โดยไม่ใช่ฟังก์ชันการสร้างหน้าต่างใหม่ แต่จะเกิดปัญหาคือ ในการเปิดไฟล์แต่ละไฟล์นั้น ขนาดของวิดีโอมีขนาดที่แตกต่างกัน แต่ต้องการแสดงผลในขนาดในกรอบที่ถูกกำหนดไว้บน Application ซึ่งขนาดของวิดีโอที่มีขนาดใหญ่เกินไปนั้น ก็ไม่สามารถแสดงผลได้ และ ขนาดที่เล็กก็เกินไป ก็ภาพที่ออกมา ก็จะเล็กมาก

ดังนั้นผู้พัฒนาโปรแกรมจึงเสนอแนวทางเป็นการสร้างฟังก์ชันขึ้นมาปรับขนาดภาพที่ได้ get มาจากวิดีโอเทียบอัตราส่วนแล้วนำมาแสดงผลบนหน้าต่างของโปรแกรม ได้ตั้งชื่อว่า DrawFunction(); โดยมี code แสดงไว้ด้านล่างนี้

```
void CMKAppDlg::DrawFunction ( HDC hDC, BYTE *pImage, LONG lDestWidth, LONG lDestHeight,
LONG lSrcWidth, LONG lSrcHeight, WORD sBitCount, DWORD biCompression)
{
    HBITMAP          hDIB;
    HDC               hMemDC;
    HPEN              hPen;
    LPBITMAPINFO      pBitmapInfo;
    if ( sBitCount >= 16 ) {
        pBitmapInfo = ( LPBITMAPINFO ) new unsigned char [ sizeof ( BITMAPINFO ) ];
    } else
    {
        pBitmapInfo = ( LPBITMAPINFO ) new char[ sizeof ( BITMAPINFOHEADER ) +
            ( 1 << sBitCount ) * sizeof ( RGBQUAD ) ]; //GrayTable[256]
        ::memcpy ( pBitmapInfo->bmiColors, &GreyTable4[0], sizeof ( GreyTable4 ) );
    }
    //pBitmapInfo->bmiHeader = BitmapInfo.bmiHeader;
    pBitmapInfo->bmiHeader.biSize      = sizeof ( BITMAPINFOHEADER );
    pBitmapInfo->bmiHeader.biWidth    = lSrcWidth;
```

```

pBitmapInfo->bmiHeader.biHeight      = lSrcHeight;
// change it when the display invert (-lSrcHeight)
pBitmapInfo->bmiHeader.biPlanes      = 1;
pBitmapInfo->bmiHeader.biBitCount     = sBitCount;
pBitmapInfo->bmiHeader.biCompression = biCompression;
pBitmapInfo->bmiHeader.biSizeImage    = 0;
pBitmapInfo->bmiHeader.biXPelsPerMeter = 0;
pBitmapInfo->bmiHeader.biYPelsPerMeter = 0;
pBitmapInfo->bmiHeader.biClrUsed      = 0;
pBitmapInfo->bmiHeader.biClrImportant = 0;

DOUBLE dDestRatio = ( double ) lDestWidth / ( double ) lDestHeight;
DOUBLE dSrcRatio  = ( double ) lSrcWidth / ( double ) lSrcHeight;

DOUBLE dZoom;
LONG   lNewDestWidth;
LONG   lNewDestHeight;

if ( dDestRatio > dSrcRatio )
{
    //shrink by height
    dZoom = ( double ) lDestHeight / ( double ) lSrcHeight;
}
else
{
    //shrink by width
    dZoom = ( double ) lDestWidth / ( double ) lSrcWidth;
}

lNewDestWidth = ( LONG ) ( 0.98 * dZoom * lSrcWidth );
lNewDestHeight = ( LONG ) ( 0.98 * dZoom * lSrcHeight );

INT dx = ( lDestWidth - lNewDestWidth ) / 2;
INT dy = ( lDestHeight - lNewDestHeight ) / 2;

hMemDC = ::CreateCompatibleDC ( hDC );
hDIB = ::CreateDIBitmap( hDC, &pBitmapInfo->bmiHeader, CBM_INIT,
    pImage, pBitmapInfo, DIB_RGB_COLORS );
::SelectObject ( hMemDC, hDIB );
SetStretchBltMode ( hDC, HALFTONE );
StretchBlt ( hDC, dx, dy, lNewDestWidth, lNewDestHeight,
    hMemDC, 0, 0, lSrcWidth, lSrcHeight,
    SRCCOPY );

hPen = ::CreatePen ( PS_SOLID, 1, RGB ( 0, 0, 255 ) );
::SetBkMode ( hDC, TRANSPARENT );
::SelectObject ( hDC, ::GetStockObject ( NULL_BRUSH ) );
::SelectObject ( hDC, hPen );
::Rectangle ( hDC, 0, 0, lDestWidth, lDestHeight );

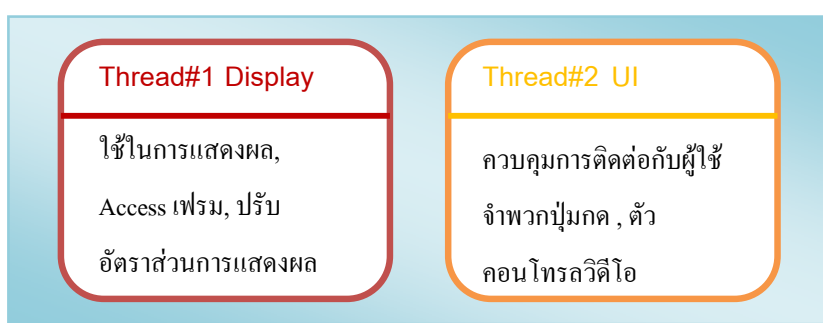
::DeleteObject ( hPen );
::DeleteObject ( hDIB );
::DeleteObject ( hMemDC );

delete [ ] pBitmapInfo;

```

6. การพัฒนาโปรแกรมในรูปแบบ Multithread

ชนิดของเธรด ซึ่งเราสามารถแบ่งเป็นสองชนิดภายใต้สถาปัตยกรรม MFC ได้แก่ เธรดงาน (Worker Thread) เป็นเธรดที่ใช้ในการคำนวณ หรือทำงานในลักษณะเป็นงานรองจากโปรแกรมหลักซึ่งจะไม่มีเมสเสจวินโดว์เข้ามาเกี่ยวข้อง แต่สามารถใช้เมสเสจที่เรากำหนดขึ้นมาได้ และเธรดที่ใช้สร้างส่วนติดต่อกับผู้ใช้ (User Interface Thread) เป็นเธรดที่ใช้ในการสร้างวินโดว์เพื่อสร้างส่วนติดต่อกับผู้ใช้งาน ดังนั้นจึงมีเมสเสจวินโดว์มากมาย โดยในโครงงานนี้ ได้แบ่งเธรดดังรูปด้านล่าง (รูปที่ 3.13)



รูปที่ 3.13 การแตกเธรดของโปรแกรมค้นหาจากโดยอ้างอิงจากผู้ใ้

การประกาศ Handle ของเธรด โดยในโครงงานมีการประกาศ 2 ตัว คือ

hGrabImageThreadHandle และ hThreadDoneEvent

DWORD WINAPI	GrabImageThread(LPVOID lpParameter);
HANDLE	hGrabImageThreadHandle;
HANDLE	hThreadDoneEvent;
DWORD	dwGrabImageThreadHandleID;

จากนั้นก็ใช้ฟังก์ชันการสร้างเธรด คือ CreateThread(); ขึ้นมา

```

if ( capture )
{
    hGrabImageThreadHandle = ::CreateThread( NULL, 0 , GrabImageThread, NULL, 0,
    &dwGrabImageThreadHandleID );
    if ( hGrabImageThreadHandle == NULL )
    {
        ::MessageBox( NULL, "Could not make a thread", NULL, MB_OK);
    }
}
  
```

```

DWORD WINAPI GrabImageThread( LPVOID lpParameter)
{
    BOOL Loop = TRUE;
    for (;;)
    {
        // display
        while ( !bDataPause && ( NumFrame > nCurrentFrame ) )
        {
            cvSetCaptureProperty( capture,
CV_CAP_PROP_POS_FRAMES, (double) nCurrentFrame );
            bDataOK = TRUE;
            frame = cvQueryFrame( capture );
            if( !frame )
                break;
            ::InvalidateRect ( hAppDlg, &DisplayRect, FALSE );
            int a = (int) cvGetCaptureProperty( capture,
CV_CAP_PROP_POS_FRAMES );
            nCurrentFrame++;
            CString text;
            text.Format ( "%d", a );
            ::SetWindowText ( hSlideNumber, text );
            pslider->SetPos ( a );
            ::Sleep ( Speedvalue );
        }
        if ( NumFrame == nCurrentFrame ) nCurrentFrame--;
        if ( bDataPause ) {
            cvSetCaptureProperty( capture,
CV_CAP_PROP_POS_FRAMES, (double) nCurrentFrame );
            frame = cvQueryFrame( capture );
            if( !frame )
                break;
            ::InvalidateRect ( hAppDlg, &DisplayRect, FALSE );
            ::Sleep ( Speedvalue );
        }
    }

    cvReleaseCapture( &capture );
    return TRUE;
}

```

7. หน้าตา User Interface ของตัวโปรแกรม



รูปที่ 3.14 การพัฒนาโปรแกรมค้นหาฉาก

1. ตัวแสดงผลวิดีโอ (Display) การแสดงผลนั้น จะสามารถแสดงผลได้เฉพาะ ไฟล์ *.AVI ได้เท่านั้น และ ตัวแสดงผลจะทำการปรับขนาดเป็นอัตราส่วนของขนาดไฟล์จริงๆ แล้วนำมาแสดงผลบนหน้าจอ ทำให้ไม่ว่าไฟล์จะมีขนาดใหญ่ หรือเล็ก ก็สามารถที่แสดงผลได้ และมี Time Line ด้านล่างตัวแสดงผล
2. เครื่องมือควบคุมวิดีโอ (Video Control) เป็นเครื่องมือ ควบคุมความเร็วในการแสดงผล (Speed) การเล่น(Play) การหยุด (Stop & Pause) โดยความเร็ว นั้น จะขึ้นอยู่กับประสิทธิภาพของเครื่องโดยตรง ดังนั้นเครื่องมือที่มีความเร็วสูง ก็จะสามารถ เร่งสปีด ได้มากขึ้น
3. เครื่องมือออกแบบรูปแบบ โดยผู้ใช้ (เฉพาะวิธีที่ 1) (Caption Control) & (Pattern) โดยจะจำลองการใช้รีโมท กับ โทรทัศน์ไว้ในส่วนนี้ และตัวโปรแกรมสามารถใช้ได้เพียงวิธีที่ 1 เท่านั้น ในวิธีที่สองนั้น ยังอยู่ในขั้นตอนการติดตั้งอยู่
4. เครื่องมือการค้นหา (Search) หลักการผ่านกระบวนการ การค้นหาฉาก ผู้ใช้สามารถที่จะกระโดดไปฉากที่ต้องการได้เลยทันที โดยใช้เครื่องมือค้นหา